Generative AI

Technical report

Group 19
Aigerim Doskozha - 01529258
Avelardo Ramirez - 12435655
Brais Guardado Collazo - 12233000
Ekaterina Grigorashchenko - 12432933
Valeriia Avakian - 12432869



TECHNISCHE
UNIVERSITÄT
WIEN

Course: Generative AI
Academic Year: WS2025
January 21, 2026

1 Abstract

We built SALVIA (Slow And Low-Value, Intelligent Agent), a RAG-powered chatbot that helps students find information in their lecture notes and study materials more easily. Students often have their notes scattered across multiple files, making it hard to find specific information when preparing for exams. We implemented two different retrieval methods backed up by their respective vector store to compare their performance: FAISS vector search and Neo4j GraphRAG. The FAISS approach uses semantic similarity to find relevant text chunks, while GraphRAG complements the information retrieved by semantic similarity with information extracted from a knowledge graph previously built from documents. We tested both systems with sensible questions related to a curated set of documents and measured how well they retrieved relevant information and generated accurate answers.

2 Description

2.1 Ideal User Journey

The student opens the SALVIA web interface and picks which retrieval method they want to use, either FAISS or Neo4j GraphRAG. After clicking the initialize button, the system loads up the knowledge base that already contains processed lecture transcripts about neural networks, transformers, reinforcement learning, and other ML topics. When the student types a question like "What is attention in transformers?", the system searches through the documents and finds the 5 most relevant chunks of text. These show up as numbered sources below the answer. The LLM then generates a response based on those sources and adds citations next to the facts it uses. Students can click on these citation badges to see the actual source text. We made sources visible by default because we wanted students to be able to verify the information themselves instead of blindly trusting the AI.

2.2 Technical Approach and Reasoning

Architecture: We use the `sentence-transformers` library and specifically the `all-MiniLM-L6-v2` embedding model to create embeddings of text chunks. These chunks are created with a recursive text splitter configured to 1200 max size and 200 characters overlap. These embeddings are then stored in a FAISS vector store, ready to be retrieved on query time. The Neo4j version does the same thing as the previous approach, but stores the embeddings within Neo4j itself and, most importantly, extracts entities and their relationships from the given chunks. This builds up a knowledge graph, and when the user asks a question, we query the graph using Cypher to find relevant nodes and their neighbours. The FAISS approach was developed first to check how a "vanilla" RAG system would perform. We added the GraphRAG approach of Neo4j to see if extracting relationships between concepts would help.

LLM Integration: We run Llama 3.2 locally through Ollama for both generating answers and extracting entities (in the GraphRAG case), but the system can be configured to use any LLM available to Ollama. The system prompt tells the model to only use information from the provided sources and to add citations when it makes a claim. Running

everything locally with Ollama means students don't need API keys or have to worry about their notes being sent to external servers, which was one of our main concerns. A drawback of this approach is, naturally, generation speed.

Interface: The web UI is built with Streamlit. The user can choose which retriever-mode to use and adjust how many sources should be used for generation. These sources are formatted below the answer after generation, and the answer itself contains clickable links to the sources supporting claims.

2.3 Challenges and Technical Obstacles

Neo4j Ingestion Times: Building the knowledge graph takes a lot of time. A timeout is set at 1500 seconds (25 min) to prevent infinite hangs. Extracting entities requires calling the LLM multiple times per chunk, and local LLM execution is not fast. To address these shortcomings, a status tracking system where each document gets marked as PENDING, OK, or FAILED was developed, to prevent re-running the pipeline for documents which have already been ingested.

Moreover, it can clearly be observed upon closer inspection of the generated knowledge graph that extracted entities and relationships are not truly helpful or novel. Fixing the possible set of possible entity and relation types was also tried, but this arguably restricts the scope of what knowledge can be represented by the KG.

Retrieval Quality: Sometimes, retrieved chunks are technically similar but not actually helpful for answering the question. We added a minimum similarity threshold (0.2) and improved the prompts to handle cases where the retrieved context doesn't contain the answer.

3 Evaluation

Table 1: Overall Performance Comparison of RAG Systems

Model	Faithfulness	Relevancy	Correctness	Context Rel.
FAISS RAG	0.84	0.92	0.72	0.70
Neo4j GraphRAG	0.84	0.90	0.60	0.84
Hybrid RAG	0.90	0.96	0.84	0.74

Table 2: Overall Average Scores Across All Metrics

Model	Overall Average
FAISS RAG	0.795
Neo4j GraphRAG	0.795
Hybrid RAG (FAISS + BM25)	0.860

Based on the metrics, Hybrid RAG performs best overall (0.860), while GraphRAG excels in context relevance (0.84).

4 Reflection

4.1 Does Your System Increase or Decrease Human Agency?

SALVIA increases human agency by supporting, not replacing, student thinking. It helps users retrieve relevant information from their own notes while maintaining control through transparent retrieval and source citations, enabling critical and independent learning.

4.2 Would You Develop This Further?

During development, we observed that similar tools already exist on the market and can achieve slightly better performance. Therefore, further development of SALVIA would only be justified if its conceptual novelty can be translated into clear and measurable performance improvements.